

M06 / STUDY COMPANION

M06 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	4
How to revise	4
DE06-01 Verify WSL2 and choose a safe Linux workspace	4
60-second recall	4
Recall task	4
DE06-02 Linux navigation and safe file operations	4
60-second recall	4
Recall task	5
DE06-03 Pipes, redirection, command history and inspection	5
60-second recall	5
Recall task	5
DE06-04 Permissions, packages, sudo and command safety	5
60-second recall	5
Recall task	5
DE06-05 Git state: init, status, add, commit, diff and log	6
60-second recall	6
Recall task	6
DE06-06 Branches, merges and real conflict resolution	6
60-second recall	6
Recall task	6
DE06-07 GitHub remotes: push, fetch, pull and tracking	6

60-second recall	6
Recall task	7
DE06-08 Pull requests and review workflow	7
60-second recall	7
Recall task	7
DE06-09 Python virtual environments inside a repository	7
60-second recall	7
Recall task	8
DE06-10 .gitignore, secrets and repository hygiene	8
60-second recall	8
Recall task	8
DE06-11 restore, revert, reflog and safe history repair	8
60-second recall	8
Recall task	8
DE06-12 Shell scripts, exit codes and environment-driven commands	8
60-second recall	8
Recall task	9
DE06-13 Repository handoff and clean-clone reproducibility	9
60-second recall	9
Recall task	9

Read this first

QUICK REVISION BOOK

Use this only after you have studied the learner book. Each lesson keeps the canonical language and core proof, plus one recall task. This is not a replacement for hands-on practice.

Coverage: 13 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to revise

Read the lesson card, close the page, explain it in your own words, then answer the recall task. If recall is weak, return to that lesson only - not the entire module.

DE06-01 | Verify WSL2 and choose a safe Linux workspace

60-second recall

A correct attempt names the actual environment rather than saying only "terminal." WSL2 is the subsystem architecture; Ubuntu is a distro; Bash is a shell. If your repo lives under /mnt/c, that is not automatically wrong, but you must know you chose the mounted Windows filesystem. A beginner can have Python installed and still be blocked by a sentence such as "run this in Linux," "use Bash," or "clone it inside WSL." Windows and Linux are not two mysterious competing computers. You can explain WSL vs WSL2 vs Ubuntu vs Bash; prove whether your distro is WSL2; identify your Linux home; and distinguish a Linux path from the mounted Windows path. You have not claimed that installation succeeded unless

[Refresh source: Microsoft Learn - Install WSL](#)

Recall task

Fresh shell + new workspace path; translate a Windows path to /mnt/c form and explain the boundary.

DE06-02 | Linux navigation and safe file operations

60-second recall

The most important repair is location awareness. If you cannot predict the target of cp/mv/rm, stop and use pwd/ls first. rm is not "move to Recycle Bin." Quoting matters when a path contains spaces. Data engineering scripts constantly create folders, inspect files, move outputs and clean temporary work. The danger is not that cd is difficult. The danger is acting in the wrong directory or using a destructive command. You can navigate without repeatedly opening File Explorer, predict relative paths, distinguish copy from move, and show exactly what an rm command will target before you run it.

[Refresh source: freeCodeCamp - Command Line Basics for Beginners \(2026\)](#)

Recall task

New source/ready tree; use one absolute and one relative path; prove copy vs move.

DE06-03 | Pipes, redirection, command history and inspection

60-second recall

| transports stdout between processes. > replaces the output file; >> appends. 2> targets stderr. If your daily output doubled because you used append on a regeneration job, the terminal syntax became a data-quality defect.

Changed retry: New text input; filter/count to stdout file and redirect a separate failure to stderr file.

Move-on rule: the changed task works with this review closed and you can explain one realistic fail A terminal becomes powerful when commands can be connected. Instead of manually opening a 50,000-line log, you can read a stream, filter it, count it and send a result to another file. You can trace a pipeline left-to-right, distinguish stdout from stderr, show the difference between > and >>, and refuse to use grep as a substitute for structured CSV parsing.

[Refresh source: freeCodeCamp - Command Line Basics for Beginners \(2026\)](#)

Recall task

New text input; filter/count to stdout file and redirect a separate failure to stderr file.

DE06-04 | Permissions, packages, sudo and command safety

60-second recall

Permission denied is evidence, not a command suggestion. Inspect owner/mode/path first. chmod u+x script.sh is a narrow change. chmod 777 gives write/execute broadly and hides the real reasoning. Linux permissions are one of the first places where beginners solve a small problem with a dangerously large hammer. "Permission denied" does not automatically mean "use sudo," and chmod 777 is not a professional universal You can read the basic rwx groups, distinguish running a script through Bash from executing it, and explain why elevated permission is a risk boundary.

[Refresh source: DigitalOcean - 50+ Essential Linux Commands \(current reference\)](#)

Recall task

Second script: remove/restore only owner execute and prove direct-execution behavior.

DE06-05 | Git state: init, status, add, commit, diff and log

60-second recall

The key model is working tree -> index/staging -> commit. git add selects the current version for the next commit; later edits are not magically restaged. git diff and git diff --staged answer different questions. Git is often taught as a spell: add, commit, push. That creates confusion as soon as git status shows three kinds of state. Git becomes much easier when you understand the three local areas first: working tree, staging and committed. Given any git status, you can point to working-tree, staged and committed state. You can prove a commit exists in git log and explain why a commit is not the same thing as a GitHub upload.

[Refresh source: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

Recall task

Stage a file, edit it again, commit, and prove the later unstaged edit remains.

DE06-06 | Branches, merges and real conflict resolution

60-second recall

A branch is a movable pointer, not a second full project folder. A conflict means Git needs a human content decision. Resolving by deleting markers without explaining the intended final behavior is not mastery.

Changed retry: Different README run-command conflict; resolve according to actual project interface.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent Branches let you develop a change without pretending the main line is already correct. Merge conflicts are not Git "breaking." A conflict means Git cannot safely decide how to combine competing edits. You can explain branch as a pointer, not a full copy; show which branch you are on; distinguish fast-forward from conflict; and resolve conflict content based on intended behavior.

[Refresh source: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

Recall task

Different README run-command conflict; resolve according to actual project interface.

DE06-07 | GitHub remotes: push, fetch, pull and tracking

60-second recall

origin is just a remote name. Fetch updates your knowledge/remote-tracking refs but does not merge into the current branch. Pull combines fetch with integration. If you used only a local bare remote, that proves Git mechanics but is not evidence that GitHub authentication/push worked.

Changed retry: Two clones + local bare remote: fetch in clone B and prove worktree does not change until integration.

Move-on rule: the changed A remote repository introduces a second copy of project history. Many beginners treat push, fetch and pull as synonyms for "sync." They are not. You need to know which direction data moves and whether your current branch You can explain origin, local branch, remote-tracking branch, push, fetch and pull without saying "they all sync." You never fabricate a GitHub push screenshot when you only used a local bare remote.

[Refresh source: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

Recall task

Two clones + local bare remote: fetch in clone B and prove worktree does not change until integration.

DE06-08 | Pull requests and review workflow

60-second recall

A PR is a hosted review proposal. The strongest evidence is a focused diff, rationale, validation and response to feedback. A solo self-review can be honest portfolio evidence, but it is not peer review. A pull request is where version control becomes a team decision. It is not "another Git command." It is a review object hosted by platforms such as GitHub: proposed branch changes, their diff, discussion, checks and merge You can distinguish branch, commit, push and PR; explain why a PR is useful even when Git can merge locally; and prove that review feedback maps to a specific diff/commit.

[Refresh source: GitHub Docs - Pull request quickstart](#)

Recall task

Changed PR/review scenario containing accidental .env; block merge and prove the proposed diff is repaired.

DE06-09 | Python virtual environments inside a repository

60-second recall

.venv is generated/disposable project environment state. It should be easy to recreate and should not be tracked. If the project only runs because of global packages, the environment contract is incomplete.

Changed retry: Second repo: recreate .venv from scratch and run project without copying environment directory.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one in A repository should describe a project, not depend on whichever Python packages happen to be installed globally on your laptop. A virtual environment gives the project an isolated package environment that can be deleted a You can explain why .venv should not be committed, prove which interpreter is active, and recreate the environment from repository instructions.

[Refresh source: Python 3.14 docs - venv](#)

Recall task

Second repo: recreate .venv from scratch and run project without copying environment directory.

DE06-10 | .gitignore, secrets and repository hygiene

60-second recall

Ignoring affects untracked matching paths; it does not purge tracked history. If a real secret was exposed, rotate/revoke it. Removing the latest file alone does not restore confidentiality. Repositories are easy to copy, fork and publish. A secret committed once can escape far beyond your laptop. .gitignore helps prevent unwanted untracked files from entering future commits, but it is not a time machine and You can explain what .gitignore does and does not do, prove ignored state, and state the correct response to an exposed real credential: rotate/revoke first, then repair repository/history as required.

[Refresh source: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

Recall task

Track fake demo-secret once; ignore is insufficient; stop current tracking and prove history still contains it.

DE06-11 | restore, revert, reflog and safe history repair

60-second recall

Choose repair based on state. Restore can discard uncommitted file changes. Revert records a new inverse commit and is friendly to shared history. Reflog is local evidence of recent reference/HEAD movement that can help locate commits. Undo commands are dangerous when chosen by memory instead of state. A professional asks two questions first: Is the bad change committed? and Has the commit been shared/published? The safest tool depends on that answer. You can choose restore vs revert based on state and explain reflog as a local recovery record, not "Git backup in the cloud."

[Refresh source: Official Git docs - reset, restore, revert and reflog](#)

Recall task

New bad shared-style commit: revert it; then create a recovery branch from a reflog-discovered commit.

DE06-12 | Shell scripts, exit codes and environment-driven commands

60-second recall

The caller sees an exit code, not your intentions. Print an error to stderr and return non-zero for failure. Quote path variables. Environment variables are configuration boundaries, and secret values should not be echoed into logs.

Changed retry: New shipment wrapper: positional input + OUTPUT_DIR; test path with spaces and distinct exit codes.

Move-on rule: the changed task works with this review closed and you can explain A shell script turns a remembered terminal sequence into an executable contract. Data pipelines and deployment jobs use exit codes to decide whether a step succeeded. A script that prints "error" but exits zero can fool a A calling script can tell success from failure using the exit status; your wrapper quotes paths; no secret is printed; and a missing required input fails before downstream work begins.

[Refresh source: freeCodeCamp - Bash Scripting Tutorial for Beginners](#)

Recall task

New shipment wrapper: positional input + OUTPUT_DIR; test path with spaces and distinct exit codes.

DE06-13 | Repository handoff and clean-clone reproducibility

60-second recall

The clean clone is the truth test. If README-only setup cannot reproduce the result, the handoff is incomplete. Hidden data, global packages, undocumented env vars and absolute personal paths are exactly what this test is designed to expose.

Changed retry: Fresh shipment repository with changed branch/conflict/revert/wrapper rules and a new clean-clone proof.

Move-on rule: the changed task works with this review closed and yo A repository is not truly reproducible because it runs on the laptop where it was created. The strongest beginner test is intentionally inconvenient: clone it into a fresh directory and follow only the documented instruct A clean clone can be set up and run from README only, the expected controls reconcile, ignored/generated files remain untracked, and you can explain the complete Git/Linux/reproducibility path in your own words.

[Refresh source: GitHub Docs + freeCodeCamp Git course](#)

Recall task

Fresh shipment repository with changed branch/conflict/revert/wrapper rules and a new clean-clone proof.